All-Pair-Shortest-Paths (APSP)

Ref: Cormen et al text book

I/p: Given a weighted directed graph $G = (V, E)$ with a weight fun $w : E \to \mathbb{R}$.

Q: find, for every pair of vertices $u, v \in V$,

a shortest weighted path from u to v

or

Declare that G contains a negative cycle

Reduction to Single-Source Shortest Paths (SSSP)

Q: How SSSP problem algorithm can be used to solve APSP problem ?

Ans:

One invocation of the SSSP subroutine will compute shortest-path distances from a single vertex s to every vertex of the graph.

Invoking the subroutine once for each of the n choices for s computes shortest-path distances for every possible origin and destination.

Looks like the Problem is already Solved.

Q: What is there to Study?

Dijkstra's Algorithm — $O(m \log n)$

↓

APSP — $O(mn \log n)$

↓

$O(n^3 \log n)$ For dense graphs

Bellman-Ford — $O(mn)$

↓

APSP — $O(mn^2)$

↓

$O(n^4)$ For dense graphs

Q: Can we design a better algorithm for APSP?

Ans YES [Floyd-Warshall Algorithm]

↓

- $O(n^3)$ algorithm even on graphs with negative edge lengths

- Uses Dynamic Programming

Basic Notation

- We use adjacency matrix representation of the graph.

- We assume that the vertices are numbered $1, 2, \ldots n$.

W – weight matrix – $n \times n$

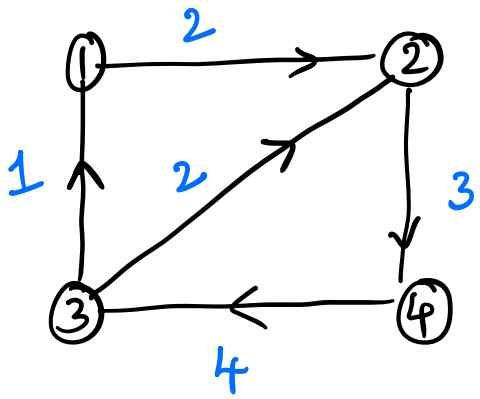
$= (w_{ij})$

$$w_{ij} = \begin{cases} 0 & \text{if } i = j \\ \text{weight of the edge} & \text{if } i \neq j,\ (i,j) \in E \\ \infty & \text{if } i \neq j,\ (i,j) \notin E \end{cases}$$

We allow negative-weighted edges

but we assume, graph has no negative weighted cycles.

Example :

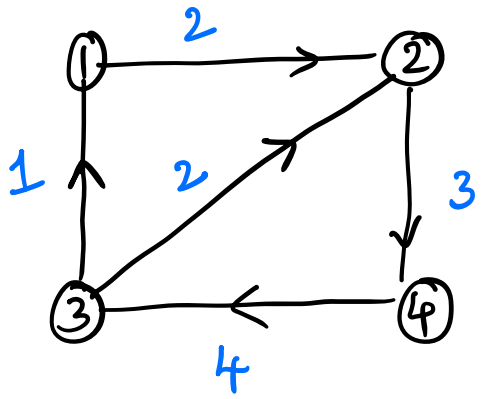


$$W = \begin{bmatrix} 0 & 2 & \infty & \infty \\ \infty & 0 & \infty & 3 \\ 1 & 2 & 0 & \infty \\ \infty & \infty & 4 & 0 \end{bmatrix}_{4 \times 4}$$

The output of the APSP algorithm is an $n \times n$ matrix $D = (d_{ij})$, where entry d_{ij} contains the weight of a shortest path from vertex i to vertex j. ie, $d_{ij} = \delta(i, j)$

$$D = \begin{bmatrix} d_{11} & d_{12} & \cdots & d_{1n} \\ \vdots & & & \\ & & & \\ d_{n1} & d_{n2} & & d_{nn} \end{bmatrix}_{n \times n}$$

Example:

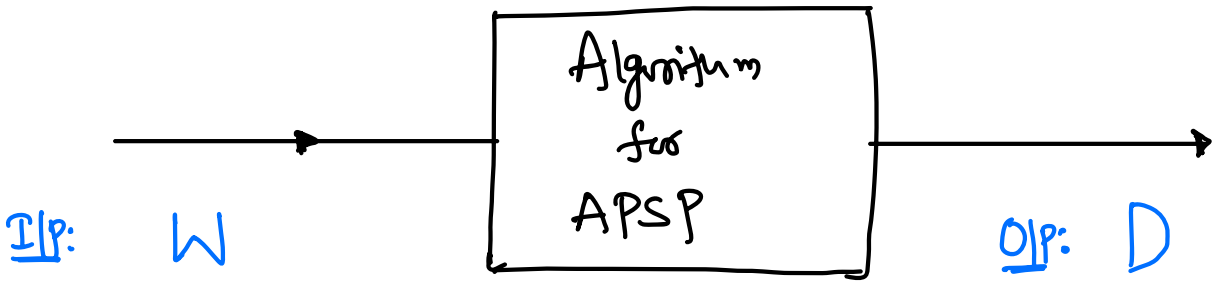


$$W = \begin{bmatrix} 0 & 2 & \infty & \infty \\ \infty & 0 & \infty & 3 \\ 1 & 2 & 0 & \infty \\ \infty & \infty & 4 & 0 \end{bmatrix}$$

Input

$$D = \begin{bmatrix} 0 & 2 & 9 & 5 \\ 8 & 0 & 7 & 3 \\ 1 & 2 & 0 & 5 \\ 5 & 6 & 4 & 0 \end{bmatrix}$$

Output



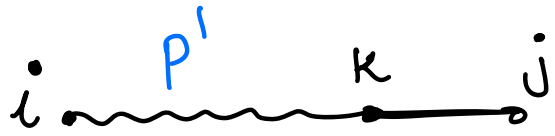
We will first discuss simpler algorithm for APSP, then we will see Floyd-Warshall Algorithm.

Structure of a Shortest Path :

Consider a shortest path p from i to j.

Suppose p contains at most m-edges.

$i \neq j$



Where p' contains at most $m-1$ edges & it is a shortest path from i to k

So $$\boxed{\delta(i,j) = \delta(i,k) + w_{kj}}$$

Let $l_{ij}^{(m)}$ → Minimum weight of any Path from i to j that contains at most m edges

$$l_{ij}^{(0)} = \begin{cases} 0 & \text{if } i=j \\ \infty & \text{if } i \neq j \end{cases}$$

Q: Recurrence for $l_{ij}^{(m)}$

For $m \geq 1$

$$\ell_{ij}^{(m)} = \min \left\{ \ell_{ij}^{(m-1)} , \min_{1 \leq k \leq n} \left\{ \ell_{ik}^{(m-1)} + \omega_{kj} \right\} \right\}$$

$$= \min_{1 \leq k \leq n} \left\{ \ell_{ik}^{(m-1)} + \omega_{kj} \right\}$$

$[\ \omega_{jj} = 0 \ \forall j\ ]$

Remark: The actual shortest-path weight from i to j given by

$$\delta(i,j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)} = \cdots$$

{ because shortest path from i to j contains at most $n-1$ edges }

Notation:

$$L^{(m)} = \left(l_{ij}^{(m)}\right)$$

$$L^{(1)} = \left(l_{ij}^{(1)}\right) = W = (w_{ij}) \quad [\text{input matrix}]$$

The final matrix $L^{(n-1)}$ contains the actual shortest-path weights.

Q: Given matrices $L^{(m-1)}$ and W

how to find $L^{(m)}$?

```
EXTEND-SHORTEST-PATHS(L, W)
1  n = L.rows
2  let L' = (l'_ij) be a new n × n matrix
3  for i = 1 to n
4      for j = 1 to n
5          l'_ij = ∞
6          for k = 1 to n
7              l'_ij = min(l'_ij, l_ik + w_kj)
8  return L'
```

Running time = $\Theta(n^3)$

```
EXTEND-SHORTEST-PATHS(L, W)
1  n = L.rows
2  let L' = (l'_ij) be a new n × n matrix
3  for i = 1 to n
4      for j = 1 to n
5          l'_ij = ∞
6          for k = 1 to n
7              l'_ij = min(l'_ij, l_ik + w_kj)
8  return L'
```

```
SLOW-ALL-PAIRS-SHORTEST-PATHS(W)
1  n = W.rows
2  L^(1) = W
3  for m = 2 to n − 1
4      let L^(m) be a new n × n matrix
5      L^(m) = EXTEND-SHORTEST-PATHS(L^(m−1), W)
6  return L^(n−1)
```



Running time = $\Theta(n^4)$.

Q: Can we improve the running time of the above algorithm by making small modifications?

Improving the Running time:

$$L^{(1)} = W$$

$$L^{(2)} = W^2 = W \cdot W$$

$$L^{(4)} = W^4 = W^2 \cdot W^2$$

$$\vdots$$

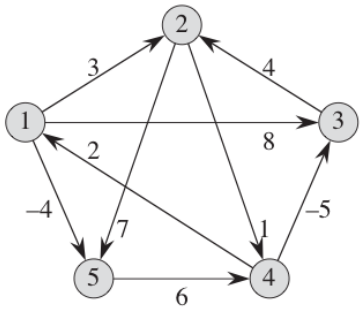
$$L^{(2^{\lceil \log(n-1) \rceil})} = W^{2^{\lceil \log(n-1) \rceil}}$$

Since $2^{\lceil \log(n-1) \rceil} \geq n-1$, the final matrix $L^{(2^{\lceil \log(n-1) \rceil})} = L^{(n-1)}$

```
FASTER-ALL-PAIRS-SHORTEST-PATHS(W)
1  n = W.rows
2  L^(1) = W
3  m = 1
4  while m < n − 1
5      let L^(2m) be a new n × n matrix
6      L^(2m) = EXTEND-SHORTEST-PATHS(L^(m), L^(m))
7      m = 2m
8  return L^(m)
```

$\Theta(n^3 \log n)$

Example :-



$$L^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \qquad L^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 2 & -4 \\ 3 & 0 & -4 & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & \infty & 1 & 6 & 0 \end{pmatrix}$$

$$L^{(3)} = \begin{pmatrix} 0 & 3 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \qquad L^{(4)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

The final iteration computes $L^{(n-1)}$ by actually computing $L^{(2m)}$ for some $n-1 \leq 2m < 2n-2$

But $L^{(2m)} = L^{(n-1)}$

FLOYD - WARSHALL ALGORITHM

Next, we study a different DP formulation to solve APSP, known as Floyd-warshall algorithm runs in $\Theta(n^3)$ time.

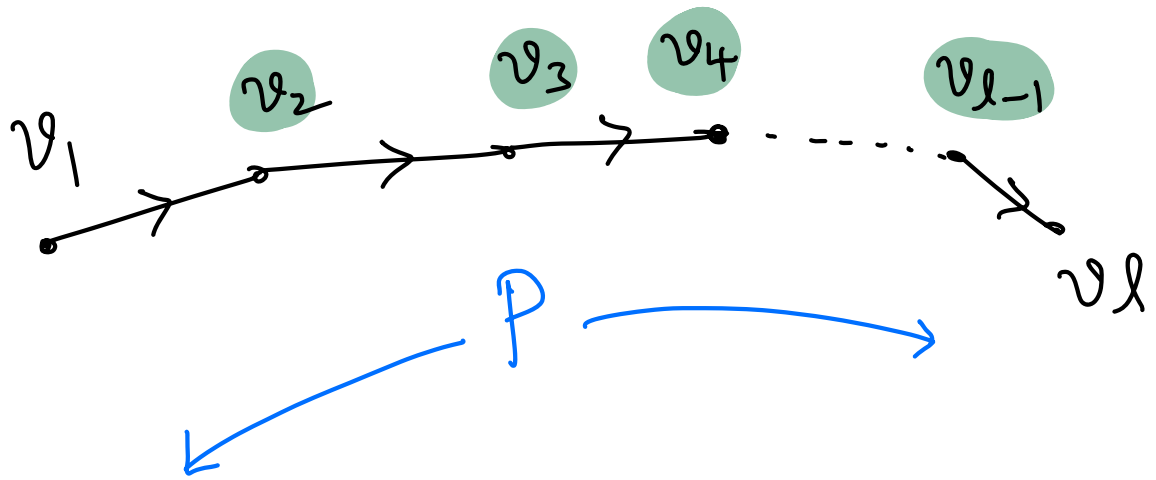
Optimal Substructure

Main Idea: $V(G) = \{1, 2, 3, \ldots n\}$

let $V^{(k)} = \{1, 2, 3, \ldots k\}$

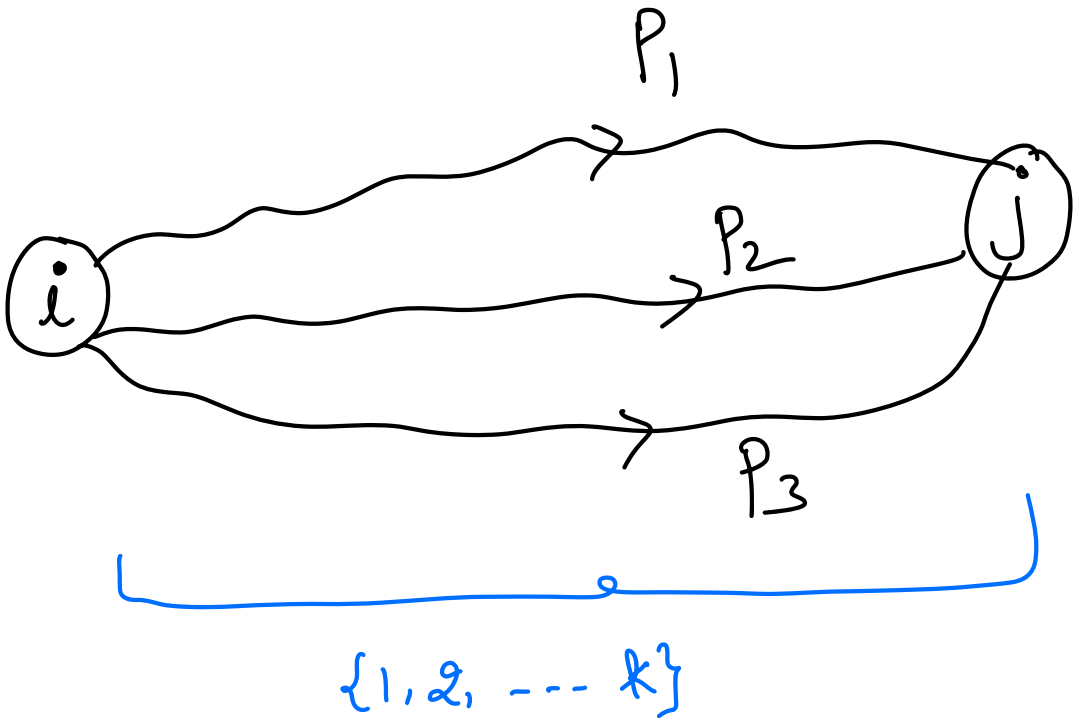
Intermediate Vertices

An intermediate vertex of a Simple Path $P = \langle v_1, v_2, \ldots v_l \rangle$ is any vertex of P other than v_1 and v_l.



Any Vertex in the set $\{v_2, \ldots v_{l-1}\}$ is an intermediate vertex of P

For any pair of vertices $i, j \in V$, consider all paths from i to j whose intermediate vertices are all drawn from $\{1, 2, \ldots k\}$.



Let P be a minimum weight path among them.

Q: What is the relationship b/w P and shortest Paths from i to j with all intermediate vertices are from $\{1, 2, \ldots k-1\}$.

Case 1: k is not an intermediate vertex of path P.

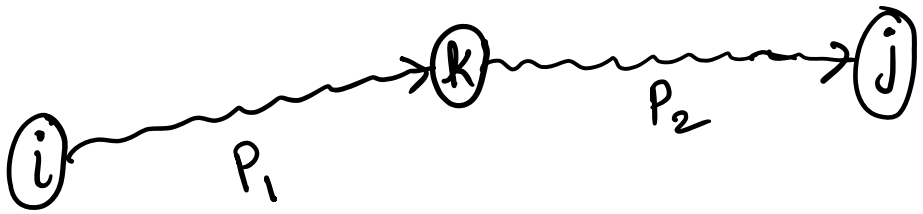
Case 1: k is not an intermediate vertex of Path P.

If k is not an intermediate vertex of path p, then all intermediate vertices of path p are in the set $\{1, 2, \ldots, k-1\}$. Thus, a shortest path from vertex i to vertex j with all intermediate vertices in the set $\{1, 2, \ldots, k-1\}$ is also a shortest path from i to j with all intermediate vertices in the set $\{1, 2, \ldots, k\}$.

Case 2: k is an intermediate vertex of Path P.

Case 2: k is an intermediate vertex of Path P.

If k is an intermediate vertex of path p, then we decompose p into $i \overset{p_1}{\rightsquigarrow} k \overset{p_2}{\rightsquigarrow} j$, as Figure illustrates. By Lemma p_1 is a shortest path from i to k with all intermediate vertices in the set $\{1, 2, \ldots, k\}$. In fact, we can make a slightly stronger statement. Because vertex k is not an intermediate vertex of path p_1, all intermediate vertices of p_1 are in the set $\{1, 2, \ldots, k-1\}$. Therefore, p_1 is a shortest path from i to k with all intermediate vertices in the set $\{1, 2, \ldots, k-1\}$. Similarly, p_2 is a shortest path from vertex k to vertex j with all intermediate vertices in the set $\{1, 2, \ldots, k-1\}$.



P: all intermediate vertices in {1, 2, .. k}

P1, P2: all " " " {1, 2, .. k-1}

DP formulation

$d_{ij}^{(k)}$ = weight of a shortest path from i to j for which all intermediate vertices are in the set $\{1, 2, \ldots k\}$

Base case :

$$d_{ij}^{(0)} = \begin{cases} 0 & \text{if } i = j \\ w_{ij} & \text{if } ij \in E(G) \\ \infty & \text{if } ij \notin E(G),\ i \neq j \end{cases}$$

Recurrence for $d_{ij}^{(k)}$

Q: Write the recursive formula for $d_{ij}^{(k)}$.

For $k \geq 1$

$$d_{ij}^{(k)} = \text{Min}\left\{ d_{ij}^{(k-1)}, \; d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \right\}$$

$d_{ij}^{(k-1)}$: Vertex k is not present on Shortest i to j Path

$d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$: Vertex k is Part of Shortest i to j Path

Final answer

For any Path, all intermediate Vertices are in the Set $\{1, 2, \ldots n\}$, the matrix $D^{(n)} = (d_{ij}^{(n)})$ gives the final answer: $d_{ij}^{(n)} = \delta(i, j)$ for all $i, j \in V$

Computing the Shortest-Path weights bottom up :

```
FLOYD-WARSHALL(W)
1  n = W.rows
2  D^(0) = W
3  for k = 1 to n
4      let D^(k) = (d_ij^(k)) be a new n × n matrix
5      for i = 1 to n
6          for j = 1 to n
7              d_ij^(k) = min(d_ij^(k-1), d_ik^(k-1) + d_kj^(k-1))
8  return D^(n)
```

Running time = $\Theta(n^3)$

(Q1) How to check if input graph G has a negative cycle?

(Q2) How to construct a shortest i-j path?

(Q2) How to construct a shortest i-j Path?

Ans (See next Page)

Idea is to compute the Predecessor of Vertex j on a shortest Path from Vertex i.

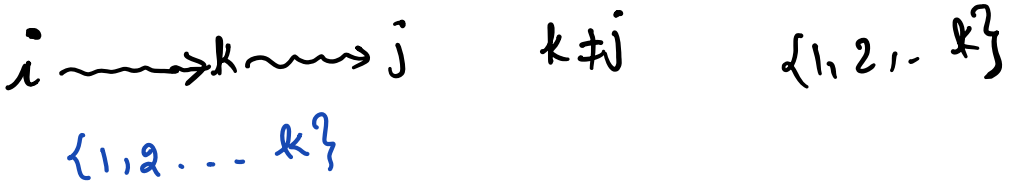
Constructing a Shortest Path:

We can compute the Predecessor matrix Π while the algorithm computes the matrices $D^{(k)}$

$$\Pi^{(0)}, \Pi^{(1)}, \text{--} \Pi^{(n)},$$

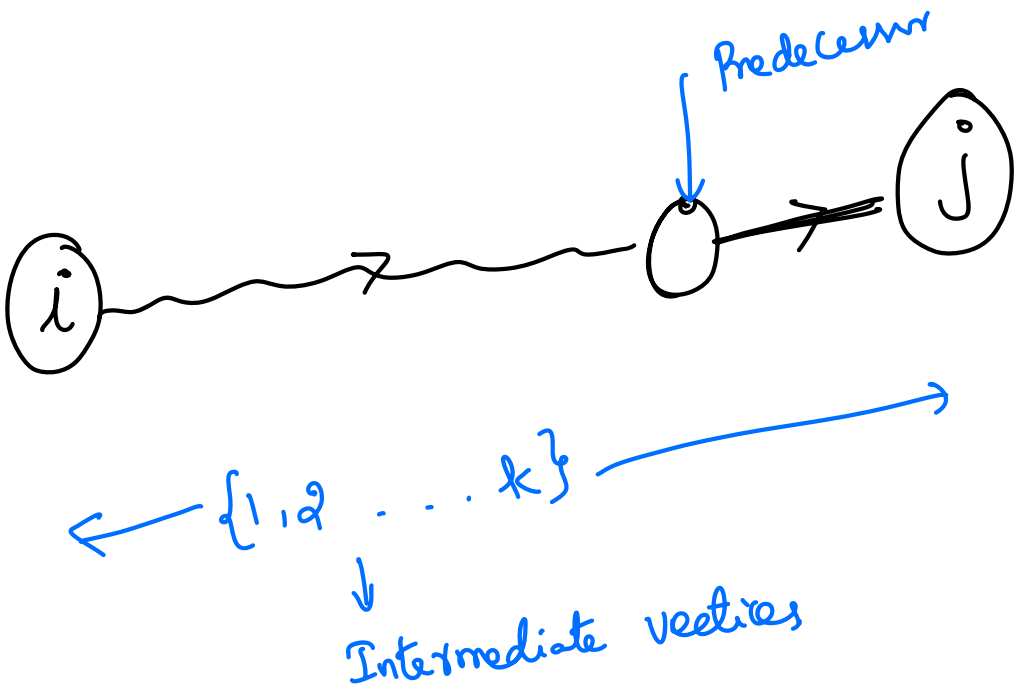
where $\Pi_{ij}^{(k)}$ as the Predecessor of vertex j on a Shortest ij Path with all intermediate vertices in the set $\{1, 2, \ldots k\}$

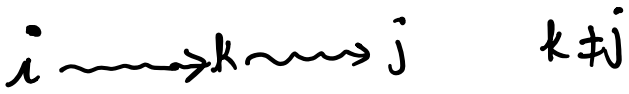
$$\pi_{ij}^{(0)} = \begin{cases} \text{NIL} & \text{if } i = j \text{ or } w_{ij} = \infty, \\ i & \text{if } i \neq j \text{ and } w_{ij} < \infty. \end{cases}$$



Predecessor of j we choose is same as the predecessor of j we choose on a shortest path from k with all intermediate vertices in the set {1, 2 . . k-1}

Q: Recursive formula for $\pi_{ij}^{(k)}$

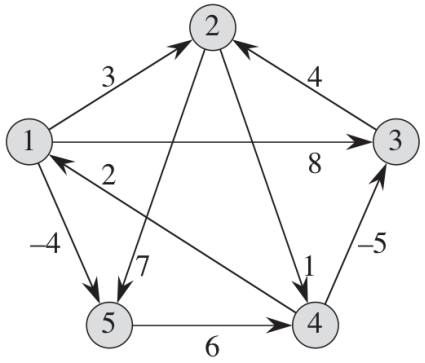




$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & \text{if } d_{ij}^{(k-1)} \le d_{ik}^{(k-1)} + d_{kj}^{(k-1)}, \\ \pi_{kj}^{(k-1)} & \text{if } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)}. \end{cases}$$

k is on shortest i-j path

Example:



$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(0)} = \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & \text{NIL} & 4 & \text{NIL} & \text{NIL} \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(1)} = \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(2)} = \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(3)} = \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(4)} = \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \quad \Pi^{(4)} = \begin{pmatrix} \text{NIL} & 1 & 4 & 2 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(5)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \quad \Pi^{(5)} = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

Exercise 1 : How to Construct Shortest Paths using the matrix $\pi^{(k)}$?

Exercise 2: How can we use the O/P of FW algorithm to detect the presence of a negative-weight cycle?

HINT: There is a negative weight cycle if and only if $d_{ii}^{(n)} < 0$ for some i

Reverse direction:

Suppose $d_{ii}^{(n)} < 0$ for some i

ie, there is a path (cycle) from i to itself, whose weight is negative. hence there is a negative weight cycle.

Forward direction:

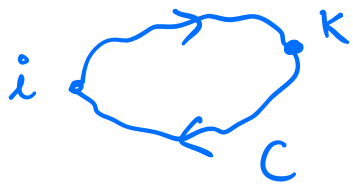
If there is a negative weight cycle,

Consider the one with fewest # of edges, (say C)

@ if C has one vertex, then some $w_{ii} < 0$

so d_{ii} starts out negative and d values are never increased, so it is negative when the algorithm terminate

(b) if C has at least two vertices, let k be the highest numbered vertex in the cycle and let i be the some other vertex in the cycle.



$d_{ik}^{(k-1)}$ and $d_{ki}^{(k-1)}$ have correct shortest path weights, because they are not based on negative-weight cycles. (Neither $d_{ik}^{(k-1)}$ nor $d_{ki}^{(k-1)}$ can include k as intermediate vertex, and i and k are on the negative weight cycle with the fewest vertices)

Since $i \rightsquigarrow k \rightsquigarrow i$ is a negative weight cycle, the sum of those weights is negative so $d_{ii}^{(k)}$ will be set to a negative value.

Since $d_{ii}^{(k)}$ values are never increased, it is also negative when the algorithm terminates.

(*) It suffices to check whether $d_{ii}^{(n-1)} < 0$ for some i.

because,

A negative cycle containing vertex i either contains n or it does not.

if doesn't then clearly $d_{ii}^{(n-1)} < 0$.

if the negative weight cycle contains vertex n, then consider $d_{nn}^{(n-1)} < 0$.